

Workshop · Module 08

Operating at *scale*.

08 / 08

Agenda

ARC 1

ENVIRONMENTS

Right key, right endpoint

- o1 Multi-env keys — DEV, STG, PROD
FUNDAMENTAL_API_KEY_{ENV} per environment.
Rotation is a variable update, not a code change.
- o2 Each environment, its own endpoint
Pass the matching api_url to Fundamental, or set
FUNDAMENTAL_API_URL. Dev and prod never share a
target.
- o3 Validate at startup — fail fast
build_validated_client calls models.list at boot. Rotation
failures surface before the first real call.

ARC 2

MODEL LIFECYCLE

Tag, then audit

- o1 client.models.set_attributes() — tag metadata
Stamp version, owner, and stage onto a trained model.
Metadata travels with the model id.
- o2 models.list() — complete inventory
Every model you have trained under the key. Materialize to
DataFrame, filter, audit.
- o3 Exercise — the production startup
Validated client, tag the model, registry write. The
capstone.



By the end of this module, you'll be able to...

Three things you'll *leave with*

A

Separate *dev, staging, prod* cleanly

Per-environment API keys (FUNDAMENTAL_API_KEY_{ENV}), a matching api_url per environment, and a validated-startup wrapper that surfaces rotation failures at boot — not at the first real call.

B

Tag trained models with *metadata*

client.models.set_attributes() stamps version, owner, and stage onto a model. Read it back with client.models.get(). The attributes dict travels with the model id.

C

Audit your model *inventory*

client.models.list() returns every trained model you own. Materialize it into a DataFrame, sort, filter by attribute. Treat model ids as production artifacts.



From *running*
to *operating*.

PART 01



The one-sentence definition.

Environment, mode, and inventory — the three operational axes that separate a working pipeline from a production service.

FIG. 08 · THE MODULE IN ONE SENTENCE



One *function*, every environment.

Per-env keys. Each environment maps to its own api_url. No code changes between dev and prod.

module_08_operating_at_scale.ipynb · cell 6

```
ENV_API_URLS = {
    "dev": "https://dev.api.fundamental.tech",
    "stg": "https://stg.api.fundamental.tech",
    "prod": "https://api.fundamental.tech",
}

def build_client(env="dev"):
    suffix = env.upper()
    key = (os.getenv(f"FUNDAMENTAL_API_KEY_{suffix}")
           or os.getenv("FUNDAMENTAL_API_KEY"))
    if not key:
        raise ValueError(f"No API key for env: {env}")
    # each environment maps to its own endpoint
    api_url = ENV_API_URLS[env.lower()]
    client = Fundamental(api_key=key, api_url=api_url)
    fundamental.set_client(client)
    return client

# usage - same code, any environment
dev_client = build_client(env="dev")
prod_client = build_client(env="prod")
```

WHY THIS MATTERS

Rotation is a variable update.

Key rotation, environment promotion, and staging deploys all become changes to environment variables — never to source. The estimator code from Modules 01 through 07 stays exactly the same.



client.models.set_attributes() — tag the model.

Metadata only, not training. Stamp version, owner, and stage onto a trained model.

● ● ● module_08_operating_at_scale.ipynb · cell 15

```
# tag a trained model with metadata — the moment fit returns  
model_id = nexus.trained_model_id_
```

```
client.models.set_attributes(model_id, {  
    "version": "1.0",  
    "owner":   "credit-risk-pod",  
    "stage":   "production",  
})
```

```
# read the metadata back from the model id  
metadata = client.models.get(model_id)  
print(metadata.attributes)
```

WHY THIS MATTERS

Metadata travels with the id.

set_attributes does not train anything — it tags the model the fit already produced. Version, owner, and stage stay attached to the model id, so future-you can identify it without reading commit history.



When row order *carries signal*.

By default the regressor treats rows as independent and may reorder them. When the sequence itself matters — a date column, a time index, an ordered log — construct it with `NEXUSRegressor(time_series=True)`. That one flag tells the platform to preserve row ordering during training and prediction.

Nothing else about the call changes. `fit(X, y)` still takes only features and targets. The order you feed in is the order the model respects.

THE RULE

*If the rows
have an
order,
keep it*

Regression with a meaningful sequence — set `time_series=True` on the constructor. It is the one documented switch for temporal data, and it preserves ordering end to end.

*order in →
order out*

row sequence
preserved
through fit and
predict.



LOOK UP AND RETIRE

Inspect a model. Prune *the stale ones*.

`client.models.get(model_id)` returns a `TrainedModelMetadata` — the model id and the attributes dict you stamped on it. Read the `stage` attribute before you load a model into production; it catches the wrong-environment id at the boundary, not at first prediction.

`client.models.delete(model_id)` permanently removes a model. Gate it on the tags you set: only prune what is labelled an experiment, never a model marked production.

THE FOUR-METHOD SURFACE

list,
get,
set_attributes,
delete

Train it, tag it, list it, look it up, retire it. Every step in the model lifecycle is a single call on `client.models` — independent of the estimator that created the model.

tag → retire lifecycle in four calls.



Three habits. *One* lifecycle.

The `client.models` surface only pays off with discipline — tag on fit, look up on deploy, prune on retire.

01

Tag on fit

Every `fit()` is followed by `set_attributes()` — version, owner, stage.
An untagged model is an orphan waiting to happen.

02

Look up on deploy

Before loading a model, call `get(model_id)` and verify `stage` is `production`. Catch the wrong-environment id at the boundary.

03

Prune on retire

`list()` the inventory, filter on the tags you set, `delete()` stale experiments. Production stays locked by its `stage` tag.



Audit the *inventory*.
Validate at *boot*.



models.list() — every model you own.

Materialize to a DataFrame. Sort, filter, audit.

```
● ● ● module_08_operating_at_scale.ipynb · cell 14

# reads FUNDAMENTAL_API_KEY / FUNDAMENTAL_API_URL from the env
client = Fundamental()
models = client.models.list()

# TrainedModelMetadata objects — not strings. Read id + attributes.
inventory = pd.DataFrame([
    "model_id": m.trained_model_id,
    "version": m.attributes.get("version"),
    "owner":    m.attributes.get("owner"),
    "stage":    m.attributes.get("stage"),
} for m in models])

# audit is just pandas — sort, filter, groupby
prod = inventory[inventory["stage"] == "production"]
print(prod.to_string(index=False))
```

WHY THIS MATTERS

Model ids are production artifacts.

`trained_model_id` is the only handle you have on a trained model. `models.list()` is how you verify your registry is complete — compare periodically and find the orphans before they become incidents.



VALIDATED STARTUP

Fail at *boot*. Not at the first call.

`build_validated_client` wraps `build_client` with one extra step: call *`client.models.list()`* before returning. It is a fast, read-only API round-trip that proves the key is valid and the endpoint is reachable.

Pair with *`initialize_production(env, model_id)`*: validate the client, load the `trained_model_id_` from your registry, return a ready-to-predict estimator. Any auth or network failure raises before your service ever handles a real request.

THE LOAD-BEARING LINE

*One
read-only
call at
boot*

`models.list()` is cheap. Running it at startup turns silent rotation failures into loud startup errors — which is exactly what your on-call rotation wants.

1 call at startup — catches key
and network issues early.



THE DISCIPLINE

Write every `model_id` *the moment* it returns.

A registry is a file — `model_id`, `task`, `mode`, `created_at`, `dataset`, `notes` — appended every time training completes. No exceptions. No "I'll write it after lunch." Every lost row is a future incident.

Reconcile against `models.list()` periodically. Anything in the inventory that is not in your registry is an *orphan* — a model someone forgot to record. Orphans do not cause incidents until someone tries to roll back to one.

THE CHECK

registry
+
models.list()
→ *no orphans*

Compare the two sets. Every model in production should be in both. Every experiment should be in the registry. Anything only in `models.list()` is a question you have not answered yet.

0 orphans the ship-it invariant
for your model fleet.



08

*The
capstone.
Stitch it all
together.*

FORMAT

Solo, at your laptop

MATERIALS

The notebook,
FUNDAMENTAL_API_KEY_DEV
set in env

OUTPUT

A validated client, a
tagged model id, a registry
line

DIFFICULTY

Advanced · ●●●●

PARTICIPANT INSTRUCTIONS

Build a validated client.
Tag a *trained* model. Write the model id
to your registry.

- 01 Set *FUNDAMENTAL_API_KEY_DEV* in your environment. Call *build_validated_client(env="dev")*. Confirm the key validates at boot.
- 02 Fit a model, capture *trained_model_id_*, then tag it with *client.models.set_attributes()* — version, owner, stage.
- 03 Append a registry line: model id, version, stage, features. *Before* you close the notebook.
- 04 Call *models.list()*, materialize the inventory DataFrame, confirm your new id appears. Paste the row into the *Your call* cell.

Three *takeaways*

If you forget everything else, remember these.

O1

Env vars only — *never* source

FUNDAMENTAL_API_KEY_{ENV} per environment, each with its own api_url. Rotation is a variable update; the code does not move.

O2

Tag every model. *Look it up on deploy.*

client.models.set_attributes() stamps version, owner, and stage. get() reads it back before you promote. The attributes travel with the model id.

O3

Validate at *startup*. Audit the inventory.

build_validated_client runs models.list at boot — catches rotation and network issues before the first real call. Treat trained_model_id_ like a production artifact.



Workshop complete.

Nine modules · from `first fit()` to a production operations playbook

YOU STARTED WITH *ONE FEATURE, ONE PREDICTION*. YOU ARE LEAVING WITH ASYNC PIPELINES, RESILIENT RETRIES, FULL DIAGNOSTICS, AND THE OPERATIONAL DISCIPLINE TO SHIP.